

Paradigmas de Programación

- Un paradigma es un enfoque o filosofía para representar y manipular elementos de un programa.
- La programación estructurada utiliza un flujo lineal de entrada-proceso-salida con datos pasivos.
- La POO organiza el software en objetos cooperativos que integran datos y métodos.
- La transición a POO cambia la terminología: los "procedimientos" pasan a ser "métodos".
- El flujo en POO se basa en el envío de mensajes entre entidades independientes.

Conceptos Fundamentales

- El paradigma se sustenta en tres conceptos clave: clase, objeto e instancia.
- **Clase:** Es una descripción abstracta, plantilla o plano que define atributos y métodos comunes.
- **Objeto:** Entidad tangible con identidad única y comportamiento definido en la memoria.
- **Instancia:** Manifestación concreta de una clase creada en un momento dado.
- **Instanciación:** Proceso de creación de un objeto utilizando el operador `new` .

Anatomía de un Objeto

- **Identidad:** Identificador único que permite al sistema rastrear cada ejemplar de forma independiente.
- **Atributos:** Rasgos o propiedades almacenados en variables que definen al objeto.
- **Estado:** Representa el valor de los atributos en un momento determinado de la ejecución.
- **Comportamiento:** Determinado por los métodos o acciones que el objeto puede realizar.
- **Interacción:** Los métodos definen cómo el objeto se relaciona con otros objetos y su entorno.

Modelado del Mundo Real

- Abstracción: Pilar que identifica rasgos esenciales ignorando detalles irrelevantes para simplificar la complejidad.
- Proceso de modelado: Identificación de entes (objetos), rasgos (atributos) y acciones (métodos).
- Ciclo de vida: Flujo que va desde el Análisis y el Diseño (UML) hasta la Implementación en Java.
- Modularidad: Facilita que el software sea flexible, mantenible y evolutivo ante cambios.

Definición de Clases en Java

- Atributos de instancia: Variables de las que cada objeto posee su propia copia independiente.
- Atributos estáticos: Variables de clase (`static`) compartidas por todos los objetos del mismo tipo.
- Métodos: Funciones que operan sobre los datos y requieren un tipo de retorno y parámetros.
- Constructor: Método especial para inicializar objetos que debe tener el mismo nombre que la clase.
- Sobrecarga de constructores: Capacidad de definir múltiples constructores con distintos parámetros en una clase.

Encapsulación y Acceso

- Encapsulación: Proceso de agrupar datos y operaciones protegiéndolos de interferencias externas.
- Caja Negra: Ocultamiento de detalles internos; el usuario solo interactúa con la interfaz pública.
- Modificadores de acceso: Determinan la visibilidad (`private`, `public`, `protected`) de los miembros.
- Getters: Métodos públicos diseñados para consultar de forma segura atributos privados.
- Setters: Métodos para modificar atributos que permiten incluir lógica de validación de datos.

Diseño de Clases Simples

- Cohesión: Principio donde cada clase debe ser responsable de una única tarea bien definida.
- Acoplamiento: Se busca que sea bajo para minimizar las dependencias directas entre clases.
- Diagrama de Clases en UML: Herramienta gráfica para visualizar la estructura de clases, sus atributos y métodos.
- Comunicación: El trabajo del sistema se realiza mediante el envío de mensajes entre objetos.

Datos y Estructuras en POO

- Tipos de datos: Java combina tipos primitivos (int, char) con tipos objeto, pero también posee tipos “envoltorio” para los tipos primitivos.
- Arrays de objetos: Colecciones que almacenan referencias a múltiples instancias de objetos en memoria.
- Colecciones dinámicas: Estructuras como `ArrayList` que permiten gestionar grupos de objetos con tamaño variable.
- Composición ("Tiene-un"): Relación donde una clase contiene objetos de otras clases como miembros.
- Ensamblaje: La composición permite la construcción de sistemas complejos mediante la unión de componentes mas sencillos.